YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

High Level Design (HLD)

Personalized Recommendations Engine for Smart City

Mohammed Shahid

Course: BSc Artificial Intelligence, Machine Learning and Data Science
Register No: 23BSAMD015
Industry Mentor: Mr.Sumit Kumar Shukla
Guided By: Ms.Fathimath Raheema U

Table of Contents

1. Introduction

SmartRec is a web-based, AI-powered decision-support system developed using Python and Streamlit. The main purpose of the system is to provide personalized urban service recommendations to citizens and visitors of Smart Cities. By leveraging Natural Language Processing and Machine Learning, SmartRec dynamically adapts its recommendations based on user context — whether the user is a daily resident, a tourist, in an emergency situation, or traveling at night.

The system ingests structured city infrastructure datasets (CSV/JSON), builds a multi-dimensional feature matrix using TF-IDF vectorization, One-Hot encoding, and normalized numeric scaling, and computes Cosine Similarity to surface the most contextually appropriate infrastructure services. A hybrid scoring formula blends content similarity with global popularity to balance relevance and reach.

1.1 Scope of the Document

This High Level Design (HLD) document defines the architecture and design decisions for SmartRec – A Context-Aware Hybrid Recommendation Framework for Smart City Infrastructure. It describes system architecture, component interactions, data flow, API design, caching, session management, and non-functional requirements. Low-level implementation details are deferred to the accompanying LLD document.

1.2 Intended Audience

This document is addressed to:

- Academic evaluators and project guides at PRJN26-230
- IBM Innovation Centre for Education industry mentors
- Developers requiring a system-level architectural reference
- QA and test engineers performing integration verification

1.3 System Overview

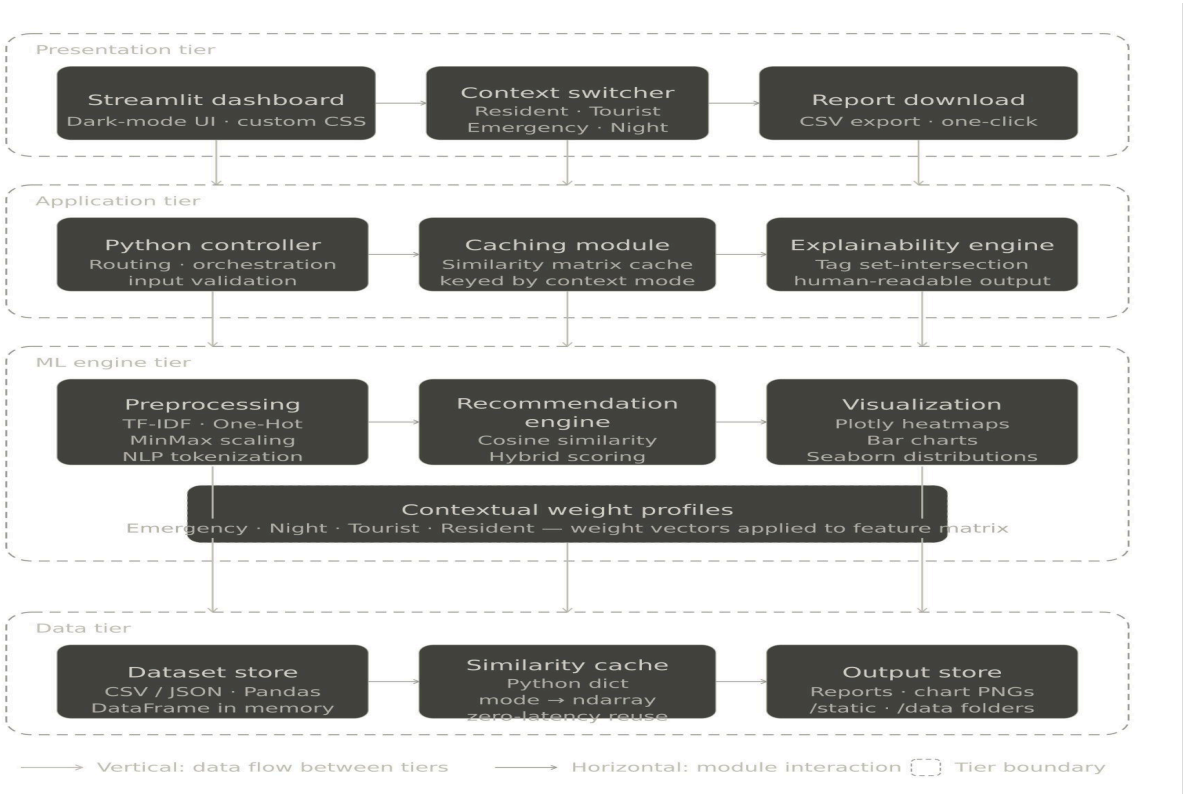
Urban environments generate enormous volumes of infrastructure and service data daily. Traditional recommendation systems fail because they rely on static popularity or simple proximity, ignoring situational needs, safety constraints, and temporal availability. SmartRec addresses these gaps with a multi-dimensional contextual recommendation engine that ingests CSV infrastructure records and produces ranked recommendations using a configurable context mode — Resident, Tourist, Emergency, or Night. Results are rendered on a dark-themed Streamlit dashboard with three visualization types, explainability output, and a downloadable CSV report.

2. System Design

2.1 Application Design

SmartRec follows a three-tier web architecture that separates presentation, application logic, and data/ML processing. The Streamlit framework serves as both the frontend and application server, rendering Jinja2-equivalent templates and handling HTTP request routing. Static assets including chart images and CSS are served from the /static folder. All data processing occurs entirely in-memory using Pandas and NumPy; no database server is required for the base deployment.

- **Presentation Tier (Streamlit / HTML / CSS):** Manages user interaction through the web interface. Allows users to select infrastructure items, toggle context modes, view recommendation results, charts, explainability outputs, and download reports.
- **Application Tier (Python / Streamlit Backend):** Acts as the central controller. Handles routing, input validation, context switching, and coordination between the Preprocessing, Engine, Visualization, and Caching modules.
- **ML Engine Tier (Scikit-learn / NumPy / Pandas):** Contains the recommendation logic. Builds TF-IDF and feature matrices, applies contextual weights, computes Cosine Similarity, and calculates Hybrid Scores to rank recommendations.
- **Data Tier (CSV / JSON / Pandas DataFrame / Local Storage):** Handles uploaded datasets, processed results, similarity cache, and downloadable CSV reports. Pandas DataFrames serve as the in-memory data store.



2.2 Process Flow

The high-level operational flow follows these steps:

1. User selects an infrastructure item and context mode through the Streamlit dashboard sidebar.
2. The Application Tier receives the selection and loads the city infrastructure dataset via load_data().
3. The Preprocessing Module applies NLP tokenization, TF-IDF vectorization, One-Hot encoding, and numeric scaling to build the combined feature matrix.
4. The Caching Module checks if a similarity matrix for the selected context already exists. If found, it is served immediately.
5. The ML Engine applies contextual weight profiles and computes Cosine Similarity across all infrastructure items.
6. Hybrid scores are calculated and top-N recommendations are ranked and returned.
7. Results are rendered on the dashboard — recommendation cards, explainability text, and Plotly similarity heatmap.

2.3 Information Flow

Information flows linearly via Streamlit's internal event model between the user interface and the backend processing modules, and through in-process function calls between system components. User selections (item name and context mode) flow from the Presentation Tier to the Application Tier as Python variables. The dataset flows from the Data Tier into the ML Engine as a Pandas DataFrame. Similarity matrices and ranked results flow from the Engine back to the Presentation Tier for rendering. All communication is in-process and in-memory — no external API calls or network requests are required.

2.4 Components Design

The SmartRec architecture maintains clear separation of concerns across six independent components, ensuring modularity, scalability, and ease of future upgrades:

- **Presentation Component (Streamlit / CSS):** Dark-mode dashboard for item selection, context toggling, KPI summary display, recommendation cards, explainability panel, and chart rendering. Replaceable with React or Angular without affecting backend.
- **Application Component (Python Controller):** Central coordinator handling context switching, function orchestration, and result routing between preprocessing, engine, visualization, and caching modules.
- **Detection / Recommendation Engine Component (ML / Statistical Layer):** Core similarity computation using TF-IDF, Cosine Similarity, contextual weighting, and Hybrid Scoring. Fully independent — upgradeable with collaborative filtering or neural embeddings.
- **Visualization Component (Plotly / Matplotlib / Seaborn):** Generates interactive heatmaps, bar charts, and distribution plots. Decoupled from detection logic — can be extended with additional chart types independently.

- **Data Component (CSV / Pandas Storage Layer):** Manages dataset loading, in-memory DataFrame operations, and downloadable CSV exports. Migratable to PostgreSQL or MongoDB without affecting engine logic.
- **Caching Component (In-Memory Dict):** Stores computed similarity matrices keyed by context mode for zero-latency repeated queries. Replaceable with Redis for distributed deployments.

2.5 Key Design Considerations

The primary design principle of SmartRec is contextual adaptability — the system must deliver meaningfully different recommendations based on the same dataset depending on who is asking and when. The following considerations guided all architectural decisions:

- Context-First Architecture: All recommendation paths are gated by a context mode selector. Changing the mode rewrites the weight vector, invalidates the cache, and forces a full similarity recomputation.
- Graceful Degradation: If the ML model file is unavailable, the system falls back to statistical Z-Score similarity. If Night Mode returns empty 24/7 results, the nearest alternatives are served.
- Explainability by Design: The system is architected to always produce a human-readable justification alongside recommendations, building user trust and academic evaluability.
- Modularity Over Monolith: Each component (preprocessing, engine, visualization, caching) is implemented as an independent Python module with defined function interfaces, enabling unit testing and independent upgrades.
- In-Memory Processing: The system avoids database overhead entirely for the base deployment, enabling fast academic and local demonstration without infrastructure dependencies.

2.6 API Catalogue

The backend exposes the following internal function endpoints and Streamlit route handlers to support dataset loading, recommendation processing, dashboard rendering, and report download:

Endpoint / Function	Type	Description
/ (Streamlit Main)	Home Route	Loads the SmartRec dashboard, renders sidebar selectors and initial KPI cards
on_item_select(item, mode)	Event Handler	Triggered on item/mode change; orchestrates preprocessing, engine, and visualization pipeline
load_data(filepath)	Data Route	Reads CSV/JSON infrastructure dataset into Pandas DataFrame; validates schema
get_recommendations(item, mode, n)	Recommendation Route	Accepts item name, context mode, and N; returns ranked top-N hybrid-scored DataFrame

Endpoint / Function	Type	Description
explain_recommendation(item_a, item_b)	Explainability Route	Returns set-intersection tag justification string for top recommendation pair
render_heatmap(sim_matrix)	Visualization Route	Generates and returns interactive Plotly heatmap of full similarity matrix
download_report(df)	Report Route	Exports current recommendation results as downloadable CSV file

3. Data Design

3.1 Data Model

The persistent data model of SmartRec is schema-flexible by design. The system accepts any CSV or JSON dataset where the required columns — item name, tags, primary_category, location_zone, popularity_score, distance_km, is_24x7, and is_accessible — are present. Uploaded files are loaded into a Pandas DataFrame and processed dynamically. During processing, derived analytical columns are appended:

Field	Type	Description
item_name	String	Unique identifier and display name of the urban infrastructure item
primary_category	String (Categorical)	Top-level category: Mobility, Healthcare, Leisure, Civic, Emergency, etc.
location_zone	String (Categorical)	City zone where the item is located: North, South, East, West, Central
tags	String (comma-separated)	Descriptive feature tags: 'mobility, electric-charging, sustainable, transit'
accessibility_features	String (comma-separated)	Accessibility attributes: 'wheelchair, braille, ramp, elevator'
popularity_score	Float64	Normalized 0–1 global popularity score based on usage metrics
distance_km	Float64	Distance in kilometres from city reference point; scaled to [0,1] during processing
is_24x7	Boolean (0/1)	Flag indicating 24-hour, 7-day operational availability
is_accessible	Boolean (0/1)	Flag indicating full accessibility compliance for differently-abled users
tfidf_vector	scipy sparse row	Derived TF-IDF feature vector for the item's tag signature (added at runtime)
hybrid_score	Float64	Derived recommendation score: (Cosine Similarity × 0.8) + (Popularity × 0.2)
final_result	String	Classification label: Recommended, Highly Relevant, or Alternative

3.2 Data Access Mechanism

All dataset access during runtime is handled through Pandas DataFrames. When a user triggers a recommendation query, the system reads the pre-loaded DataFrame from memory, applies column-level filtering and vectorization, and returns processed results within the same request cycle. No external database queries or disk reads are performed after the initial load. The similarity matrix, once computed for a context mode, is stored in the in-memory Similarity Cache (Python dict) and reused for subsequent queries against the same mode.

3.3 Data Retention Policies

Given the academic and local deployment scope of SmartRec, data retention is managed through local file storage. Uploaded datasets, generated recommendation reports, chart images, and model files are stored in designated local directories (/data, /static, /models) for runtime use. No automated long-term retention, archival, or purge policies are implemented in Version 1.0. Each new upload overwrites the previous dataset. Generated reports are overwritten on each new recommendation query.

3.4 Data Migration

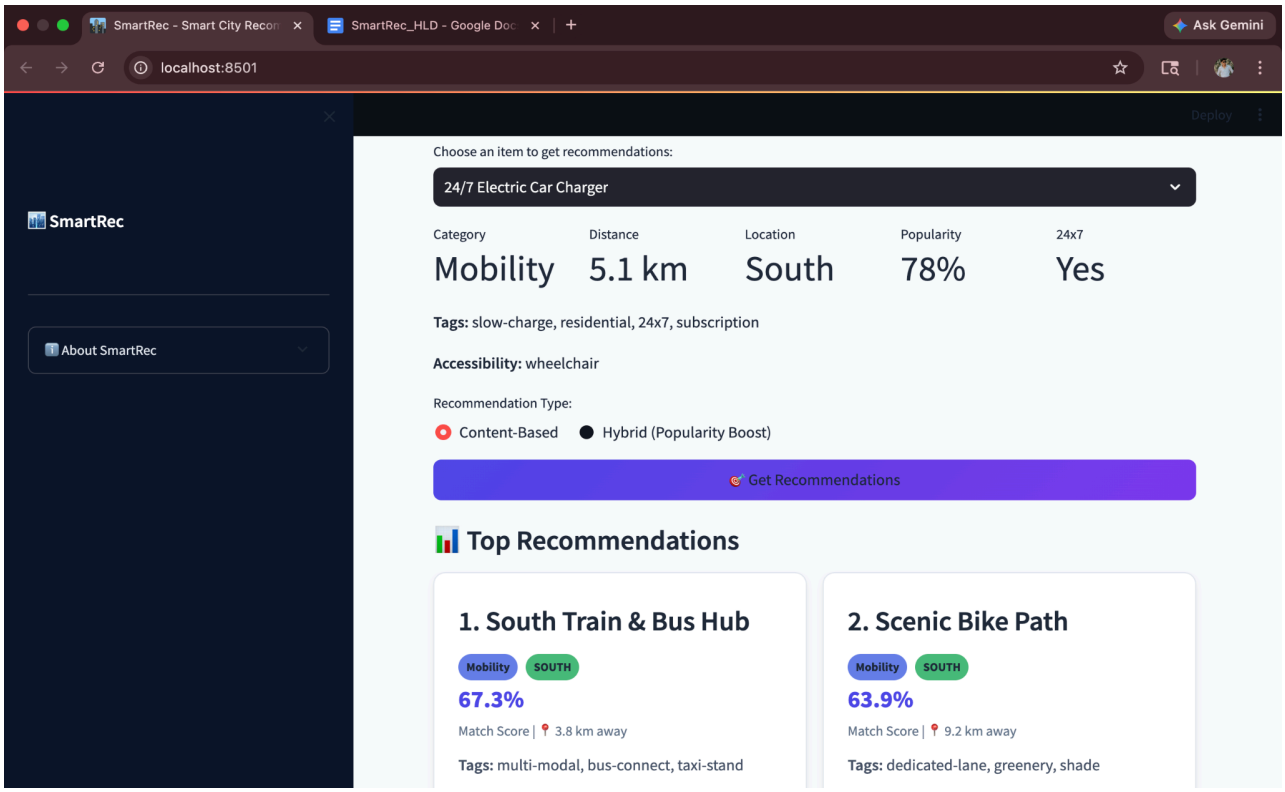
SmartRec Version 1.0 does not require an automated data migration framework because the system does not rely on a persistent relational database schema. Data processing is file-based — user-provided CSV/JSON datasets are read dynamically at runtime and processed entirely in-memory via Pandas DataFrames. Future versions targeting cloud or enterprise deployment may migrate to PostgreSQL or MongoDB, at which point a schema migration tool (e.g., Alembic) would be introduced without impacting the recommendation engine logic.

4. Interfaces

SmartRec provides a unified web-based interface for infrastructure recommendation, explainability, and report analysis through a single Streamlit dashboard. The interface is designed for both technical and non-technical users, allowing item selection, context configuration, real-time recommendation review, visualization exploration, and report download without requiring programming knowledge.

Anomaly Detection / Recommendation Dashboard

The SmartRec dashboard combines data configuration, KPI summaries, recommendation cards, explainability output, and chart visualizations in one responsive dark-mode screen.



User Interaction Features

- **Infrastructure Item Selector:** Dropdown allowing users to choose any city service or infrastructure item from the loaded dataset for recommendation querying.
- **Context Mode Toggle:** Four-option selector (Resident / Tourist / Emergency / Night) that dynamically reconfigures the contextual weight profile and triggers a new similarity computation.
- **KPI Summary Cards:** Real-time display of Total Infrastructure Items, Top Match Hybrid Score, Active Context Mode, and Number of Shared Features with top recommendation.
- **Recommendation Cards:** Visual cards for each top-N result displaying item name, category, zone, distance, popularity score, and hybrid score.

- **Explainability Panel:** Auto-generated human-readable justification text citing shared tags, matched categories, and zone proximity between the selected item and top recommendation.
- **Visualization Tab:** Interactive Plotly heatmap of the full N×N city infrastructure similarity matrix, enabling urban planners to identify service clusters.
- **Report Download Interface:** One-click button to export the current recommendation results and metadata as a downloadable CSV file.

5. State and Session Management

The SmartRec Application Tier operates in a stateless-per-request manner. Each user interaction (item selection or mode change) triggers an independent processing pipeline without dependency on memory from previous interactions. However, Streamlit's built-in session_state mechanism is used to persist UI state across widget interactions within the same browser session, preventing redundant recomputation on minor UI events.

State Handling Mechanism

- **Per-Interaction Processing:** Every context mode change or item selection independently triggers the full preprocessing → engine → visualization pipeline.
- **Streamlit Session State (st.session_state):** Used to retain the currently loaded DataFrame, active context mode, and last computed similarity matrix across widget re-renders within the same session.
- **Similarity Cache as State Store:** The in-memory similarity cache (Python dict keyed by mode) acts as a persistent state store for computed matrices, surviving across multiple user queries within the same server session.
- **File-Based Latest State:** Generated outputs such as recommendation_report.csv and chart PNG files represent the latest computed state and are overwritten on each new query.
- **No User Authentication Sessions:** SmartRec Version 1.0 does not implement user login, authentication, or per-user session isolation. All state is shared within a single server process.
- **Shared Runtime Model:** If a pre-trained ML model (e.g., Isolation Forest pickle) is loaded at startup, it remains in application memory as a read-only shared object across all recommendation requests.

6. Caching

To improve runtime efficiency and eliminate redundant computation, SmartRec uses a multi-level lightweight caching strategy. The caching architecture is intentionally simple and sufficient for local and academic deployment scope, with a defined upgrade path to Redis for distributed production environments.

- **Similarity Matrix Cache (In-Memory Dict):** Once a Cosine Similarity matrix is computed for a given context mode (e.g., Emergency, Night), it is stored in a Python dictionary keyed by mode name. Subsequent recommendation queries for the same mode retrieve the cached matrix instantly, achieving zero-latency response after the first computation.
- **TF-IDF Vectorizer Cache (Streamlit @st.cache_data):** The TF-IDF vectorizer and the resulting sparse matrix are cached using Streamlit's @st.cache_data decorator. This prevents re-vectorization on every UI re-render, significantly reducing processing time for large datasets.
- **Dataset Cache (Streamlit @st.cache_data):** The loaded and preprocessed Pandas DataFrame is cached at session level to avoid repeated disk reads and CSV parsing on each widget interaction.
- **ML Model Cache (In-Memory Object):** The pre-trained ML model, if available, is loaded once at application startup and retained in memory as a read-only object, eliminating repeated pickle deserialization overhead.
- **Static Chart Cache (File-Based):** Generated chart PNG files are stored locally in the /static folder and served directly by Streamlit's static file handler. Browser-level HTTP caching reduces re-download overhead.
- **Cache Invalidation:** The similarity cache is invalidated and cleared when the user uploads a new dataset or when an explicit cache-clear action is triggered. Context mode changes do not clear the full cache — only the entry for the new mode is computed if missing.
- **Future Upgrade Path:** For multi-user production deployments, the in-memory Python dict cache can be replaced with a Redis instance without changing the engine logic, as the cache interface is abstracted behind get_cache() and set_cache() function calls.

7. Non-Functional Requirements

7.1 Security Aspects

- **Input Validation:** Uploaded files are validated for correct extension (CSV/JSON) and required column presence before processing begins. Invalid or malformed files are rejected with user-friendly error messages.
- **File Upload Protection:** Uploaded datasets are stored in a controlled local /data directory. Future production versions should implement secure filename sanitization, upload size limits, and antivirus scanning.
- **Error Handling for Invalid Files:** If no file is selected, an unsupported format is uploaded, or the dataset is empty or corrupted, the system stops processing safely and returns a clear error message without crashing.
- **Numeric Column Validation:** If no numeric columns are detected in the uploaded dataset, recommendation processing is halted and the user is notified to provide a valid analytical dataset.
- **Exception Handling:** All runtime exceptions during CSV reading, matrix computation, chart generation, or similarity lookup are caught using try-except blocks, preventing system termination and exposing only controlled error messages.
- **Path Traversal Prevention:** Future versions should sanitize uploaded filenames using secure_filename() or equivalent to prevent directory traversal attacks.
- **Model File Safety:** The ML model pickle file is treated as a trusted internal artifact. Production deployments should implement checksum verification and restricted write access.
- **Debug Mode Restriction:** Streamlit debug mode must remain disabled in production deployments to prevent internal stack trace leakage.
- **Future Security Enhancements:** Authentication, HTTPS/TLS termination, audit logging, role-based access control, and API key management should be added for enterprise deployments.

7.2 Performance Aspects

SmartRec is architected for responsive performance on small to medium-sized infrastructure datasets (up to 10,000 items) while maintaining accurate similarity computation and context switching.

- **Fast In-Memory Processing:** Uploaded CSV/JSON files are loaded directly into Pandas DataFrames, enabling quick filtering, aggregation, and similarity computation without database latency.
- **Sparse Matrix Efficiency:** TF-IDF vectorization produces scipy sparse matrices, keeping memory usage low even for large tag vocabularies. For 10,000 items with 500-dimensional TF-IDF vectors, memory footprint remains under 50MB.
- **Zero-Latency Cached Queries:** After the first computation for a context mode, all subsequent recommendation queries for that mode are served from the in-memory cache with sub-millisecond latency.
- **Single-Pipeline Request Handling:** Each recommendation request completes preprocessing, similarity computation, hybrid scoring, explainability generation, and chart rendering within a single synchronous pipeline.

- **Selective Column Processing:** Only relevant feature columns (tags, category, zone, numerics) are processed per query, avoiding full-dataset iteration and reducing computational overhead.
- **Interactive Chart Performance:** Plotly heatmaps are rendered client-side in the browser using WebGL acceleration, maintaining smooth interactivity even for large N×N similarity matrices.
- **Concurrent User Scope:** Streamlit's default server is suitable for up to 10 concurrent users. For higher concurrency, deployment via Gunicorn with multiple workers is recommended.
- **Scalability Path:** Future throughput improvements include asynchronous task queues (Celery), background chart generation, multi-column parallel similarity computation, and Redis-backed distributed caching.

8. References

- Low-Level Design (LLD): SmartRec – Context-Aware Hybrid Recommendation Framework (Version 1.0) — Contains internal module architecture, function signatures, data structure definitions, error handling strategy, and interface specifications.
- Algorithm Literature: Isolation Forest — Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. IEEE International Conference on Data Mining. Used as optional ML-based anomaly/outlier detection within the recommendation engine.
- Algorithm Literature: TF-IDF — Salton, G., & McGill, M. J. Introduction to Modern Information Retrieval. Used for semantic tag vectorization of city infrastructure descriptions.
- Algorithm Literature: Cosine Similarity — Manning, C. D., Raghavan, P., & Schütze, H. Introduction to Information Retrieval. Core similarity metric of the recommendation engine.
- Framework Reference: Streamlit Official Documentation — https://docs.streamlit.io. For routing, session state, caching decorators, and widget rendering.
- Data Processing Reference: Pandas Official Documentation — https://pandas.pydata.org/docs. For DataFrame operations, CSV reading, filtering, and export.
- Numerical Computing Reference: NumPy — https://numpy.org/doc. For vector operations, matrix concatenation, and similarity calculations.
- ML Reference: Scikit-learn — https://scikit-learn.org. Powers TfidfVectorizer, cosine_similarity, and MinMaxScaler.
- Sparse Matrix Reference: SciPy — https://docs.scipy.org. Used for memory-efficient sparse matrix operations on TF-IDF outputs.
- Visualization Reference: Plotly — https://plotly.com/python. Used for interactive similarity heatmaps and bar charts.
- Visualization Reference: Matplotlib / Seaborn — https://matplotlib.org. Used for data distribution and exploratory analysis plots.
- Security & Deployment Reference: Internal development guidelines for secure file upload handling, local deployment configuration, and production hardening best practices.